Procedural Music System — Modification Guide

File Map

```
Scripts/
  Core/TonalPitchSpace.cs            Music theory engine
  Composition/CompositionEngine.cs   Note/rhythm decisions for all
instruments
  Synthesis/SynthEngine.cs           Audio generation (oscillators, filters,
etc.)
  Synthesis/VoiceManager.cs          Instrument presets, mixer, effects
  Bridge/ProceduralMusicController.cs  Unity API, state configs, game
interface
  Examples/FullSystemTest.cs         IMGUI test panel
  Examples/MusicTestController.cs    Keyboard controls
```

Instruments (11 total)

Index	Name	Type	Vol	Used in
0	HurdyGurdy	HurdyGurdy	0.22	Most states (drone pad)
1	Flute	LegatoLead	0.35	Melody voice
2	Bass	Subtractive	0.38	Grounding harmony
3	Log Drum	Percussion	0.85	Folk kick
4	Frame Drum	Percussion	0.45	Folk snare

Index	Name	Type	Vol	Used in
5	Brush	Percussion	0.18	Folk hi-hat
6	Folk Strings	StringEnsemble	0.22	Harmonic support
7	Kantele	PluckedString	0.7	Folk arpeggios
8	Sub Drone	Subtractive	0.55	Horror only
9	Shriek String	StringEnsemble	0.2	Horror only
10	Banjo	PluckedString	0.5	Cozy only

Change Instrument Volumes

File: `VoiceManager.cs` — static `InstrumentPreset` properties at the top.

Find the preset, change `Volume`:

```csharp
public static InstrumentPreset Banjo => new InstrumentPreset
{
    // ...
    Volume = 0.5f   // Change this (0-1)
};
```

Game States (8 total)

State	Mode	Tempo	Distortion	Vinyl	Tritones	Style
Exploring	Dorian	78-100	—	—	—	Folk
Exploring2	Major	90-112	—	—	—	Triumphant
Pressure	Nat. Minor	85-110	✓	—	—	Tense
Combat	Harm. Minor	85-110	✓	—	—	Tense
Spooky	Phrygian	55-75	✓	—	✓	Sparse
Horror	Locrian	40-20	✓	✓	✓	Horror
Night	Aeolian	55-85	✓	—	—	Sparse
Cozy	Major	62-78	—	—	—	Folk

Change State Parameters

File: `ProceduralMusicController.cs` — inside `MusicStateConfig.GetDefault()`.

Each state config has these fields:

Field	What it does
`PreferredMode`	Scale (Major, NaturalMinor, HarmonicMinor, Dorian, Mixolydian, Aeolian, Phrygian, Locrian)
`TempoMin/Max`	BPM range, interpolated by tension
`BaseTensionOffset`	Added to game tension
`MaxTension`	Ceiling for chord selection only (not layer entry)
`RootOffset`	Semitones from StartingKey (0=same, 7=fifth, -3=minor 3rd down)
`EnableDistortion`	Distortion scales with tension
`DistortionIntensity`	Max distortion (0-1, default 0.6)
`EnableVinyl`	Broken record effect
`VinylIntensity`	How broken (0-1)
`AllowTritones`	Adds diminished/augmented/tritone-sub chords
`MusicStyleSetting`	Sets ALL instrument styles at once
`FMModIndexMultiplier`	Timbral brightness
`BassRootOnly`	Bass only plays root note
`Pad` through `Banjo`	LayerRange per instrument

Add a New State

1. Add to enum in `ProceduralMusicController.cs`:

```csharp
public enum GameMusicState { ..., YourState }
```

2. Add config in `GetDefault()`:

```csharp
case GameMusicState.YourState:
    return new MusicStateConfig
    {
        State = state,
        PreferredMode = MusicalMode.Dorian,
        TempoMin = 80, TempoMax = 100,
        BaseTensionOffset = 0f, MaxTension = 0.7f, RootOffset = 0,
        Pad = LayerRange.Always,
        Kantele = LayerRange.Always,
        Melody = LayerRange.From(0.15f),
        // ... set all layer ranges
        MusicStyleSetting = MusicStyle.Folk
    };
```

3. Call: `music.SetGameState(GameMusicState.YourState);`

Layer Ranges

Controls when each instrument plays based on LayerTension:

```csharp
LayerRange.Always                // Always on (0 to 1)
LayerRange.Off                   // Never plays
LayerRange.From(0.4f)            // Enters at tension 0.4, stays on
LayerRange.Until(0.5f)           // On at start, drops out at 0.5
LayerRange.Between(0.2f, 0.7f)   // Only between 0.2 and 0.7
```

Set per-state per-instrument in each state config. Examples:

```csharp
// Kantele plays at low tension, fades as things get scary
Kantele = LayerRange.Until(0.45f),

// Percussion only at high tension
Percussion = LayerRange.From(0.6f),

// Flute plays in the mid range only
Melody = LayerRange.Between(0.1f, 0.6f),
```

Important: LayerTension is NOT clamped by MaxTension. MaxTension only affects chord selection. So instruments won't get cut off by the harmonic ceiling.

Music Styles

Each state sets a MusicStyle that controls rhythm patterns for ALL instruments:

Style	Melody	Bass	Percussion	Strings	Kantele
Folk	Dotted, triplets, lilting	Walking quarters	Groovy log drum	Warm sustained	Flowing arpeggios
Tense	Syncopated eighths	Driving, syncopated	Driving hi-hats	Rhythmic pulsing	Tremolo picking
Sparse	Long held notes, silence	Whole/half notes	Sparse	Ghostly sustained	Single lonely pluck
Triumphant	Bold quarters, march	March feel	Bold downbeats	Chords on 1+3	Full chord strums
Horror	Rare broken phrases	Root only	(not used)	Dissonant semitone	Dissonant pluck

Set All Styles at Once

```csharp
MusicStyleSetting = MusicStyle.Folk  // Sets melody, bass, percussion, strings,
```

Set Per-Instrument Styles

In the state config, after `MusicStyleSetting`, override individual styles:

```csharp
MusicStyleSetting = MusicStyle.Folk,    // Sets all
StringsStyle = MusicStyle.Tense,        // Override just strings
```

In the bridge, per-instrument fields: `MelodyStyle`, `BassStyle`, `PercussionStyle`, `StringsStyle`, `KanteleStyle`

Change Melody (Flute) Behavior

File: `CompositionEngine.cs` — `MelodyGenerator` class.

Tuning Fields

```csharp
OctaveMin = 4              // Range (both 4 = one octave)
OctaveMax = 4
RestProbability = 0.15     // 15% chance of rest
StepwiseMotionBias = 1.5   // Strong preference for small intervals
SyncopationAmount = 0.12   // Slight timing shifts
TiedNoteProbability = 0.2  // 20% chance of holding notes longer
LeapRecoveryBias = 0.9     // Step back after big jumps
```

Chord Strictness (Free vs Locked)

In `BuildCandidates()`:

```csharp
float chordStrictness = Mathf.Clamp01((tension - 0.2f) / 0.5f);
```

- Below 0.2: free — scale tones available, flute can wander
- Above 0.7: locked — sticks to chord tones, blends with ensemble
- Always free: `float chordStrictness = 0f;`

- Always strict: `float chordStrictness = 1f;`

Rhythm Patterns

In `SelectRhythmPattern()`, switch on `Style`. Each style has its own pool. Patterns are in `_rhythmPatterns[]` — arrays of beat positions in a 4-beat measure.

Change Bass Rhythm

File: `CompositionEngine.cs` — `BassGenerator` class.

Patterns in `_bassPatterns[]`, selected by style in `SelectBassPattern()`. Pitch selection in `ChooseBassPitch()`.

Set `BassRootOnly = true` in a state config to lock bass to root note only.

Change Kantele Patterns

File: `CompositionEngine.cs` — inside `GenerateMeasure()`, search for `kanteleActive`.

Switch on `CurrentKanteleStyle` with patterns for each MusicStyle. Change `spacing` for speed, modify `kantelePitches` for different notes.

Change Banjo Patterns

File: `CompositionEngine.cs` — inside `GenerateMeasure()`, search for `banjoActive`.

Three fingerpick roll patterns:

- **Forward roll:** root→mid→high→root→mid→high→root→high

- **Alternating:** root→high→mid→high→root→high→mid→high
- **Sparse:** root→high→root→mid (half speed)

Change the `rollPatterns` arrays or note assignments to customize.

Change Instrument Sounds

File: `SynthEngine.cs`

Type	Key Parameters
HurdyGurdy	`GenerateHurdyGurdy()`: buzz = `* 0.2f`, wobble Rate/Depth, filter cutoff
LegatoLead	`GenerateLegatoLead()`: partial amplitudes (1.0, 0.12, 0.18, 0.05), `_fluteBreathAmount`
Subtractive	Saw + sub-sine, preset `FilterCutoff`/`FilterEnvAmount`
Percussion	Per MIDI note in NoteOn: `_percPitchEnv`, `_percPitchTarget`, `_percSineAmt`, `_percNoiseAmt`
StringEnsemble	4 detuned saws, preset filter/vibrato settings
PluckedString	`_pluckDamping` (sustain), preset `FilterCutoff` (brightness)

Add a New Instrument

1. **Preset** in `VoiceManager.cs`:

```csharp
public static InstrumentPreset MyInst => new InstrumentPreset
{
    Name = "MyInst", SynthType = SynthVoice.SynthType.PluckedString,
    MaxPolyphony = 4, Attack = 0.01f, Decay = 1.0f, Sustain = 0f, Release = 0.5
    FilterCutoff = 3000f, Volume = 0.5f
};
```

2. **Register** in `ProceduralMusicController.cs` `Initialize()`:

```csharp
_myVoices = _mixer.AddInstrument(InstrumentPreset.MyInst); // index 11
```

3. **Index + Range** in `CompositionEngine.cs`:

```csharp
public int MyInstIndex = 11;
public LayerRange MyInstRange = LayerRange.Off;
```

4. **Generate notes** in `GenerateMeasure()`:

```csharp
```

```
if (MyInstRange.IsActive(lt))
{
    _pendingNoteOns.Add((measureStart,
        new NoteEvent(midiNote, vel, duration, MyInstIndex, 0f)));
}
```

5. **State config** — add `public LayerRange MyInst = LayerRange.Off;` to `MusicStateConfig`, enable in desired states.
6. **Wire** in `ApplyStateConfig()`:

```
csharp

_composer.MyInstRange = config.MyInst;
```

7. **Test GUI** — add name to `_instrumentNames` in `FullSystemTest.cs`.

Effects

Distortion

Per-state: `EnableDistortion = true`, `DistortionIntensity = 0.6f` Runtime: `music.SetDistortion(true);` Scales 0→intensity with tension. Atan waveshaper + bitcrushing above 0.5.

Vinyl (Broken Record)

Per-state: `EnableVinyl = true`, `VinylIntensity = 0.65f` Wow/flutter (pitch wobble), random crackle pops, LP frequency roll-off. Currently used in Horror only.

Reverb

File: `VoiceManager.cs` — `MasterMixer` fields:

```csharp
_reverbDelay    = 0.05f   // Seconds
_reverbFeedback = 0.4f    // 0-0.95
_reverbMix      = 0.15f   // Wet/dry
```

Change Key at Runtime

```csharp
music.SetKey(PitchClass.D);                                    // Root only
music.ForceModulation(PitchClass.Fs, MusicalMode.HarmonicMinor); // Root + mode
```

`SetKey()` updates `StartingKey` so future state changes calculate RootOffset from the new key.

Tension Flow

```
Game Code: SetTension(0.8)
    |
    ▼
Bridge: effectiveTension = 0.8 + BaseTensionOffset
    |
    ├──▶ TensionTarget = min(effective, MaxTension)
    |       └── Chord selection via TPS (clamped, stays musical)
```

```
    │
    ├──▶ LayerTension = effective
    │       └── LayerRange.IsActive() checks (unclamped, instruments
enter/exit freely)
    │
    ├──▶ DistortionAmount = lerp(0, DistortionIntensity, effective)
    │       └── Waveshaper drive (when EnableDistortion = true)
    │
    └──▶ VinylAmount = lerp(0, VinylIntensity, effective)
            └── Wow/flutter/crackle (when EnableVinyl = true)
```

Public API

csharp

```
// Core
SetTension(float 0-1)
SetGameState(GameMusicState)
SetKey(PitchClass)
ForceModulation(PitchClass, MusicalMode)
SetDistortion(bool)

// Layers
SetLayerEnabled(bool pad, melody, bass, perc, strings, kantele)
SetLayerRanges(LayerRange pad, melody, bass, perc, strings, kantele)

// Info
GetCurrentMusicalTension() → float
GetCurrentChord() → Chord
GetInstruments() → List<VoiceManager>
GetComposer() → CompositionEngine
DspAvgMs, DspPeakMs, DspCpuPercent, DspBudgetMs  // CPU profiling

// Emergency
Panic()
```